



บทที่ 7

คอนสตรัคเตอร์ คลาสไม่สมบูรณ์และอินเตอร์เฟส

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรรยาโรจน์

หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟส (Interface)



หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟส (Interface)



คอนสตรัคเตอร์ (constructor)

คอนสตรัคเตอร์ในภาษาจาวาเป็นเมธอดพิเศษ (Special Method) ที่ถูกเรียกใช้อัตโนมัติเมื่อมีการสร้างอ็อบเจกต์จากคลาสนั้นๆ วัตถุประสงค์หลักของคอนสตรัคเตอร์ คือ (1) การกำหนดค่าเริ่มต้น (initialize) ให้กับอ็อบเจกต์ หรือ (2) การดำเนินการเริ่มต้นอื่นๆ ที่จำเป็น

```
public class Person{
    private String name;
    ① public Person () {}
    ② public Person (String name) {
        this.name = name;
    }
}
```

กำหนดให้ object → constructor
" class → static

ถูกบังคับให้ auto เมื่อมีการสร้าง object

โดยที่ (1) คอนสตรัคเตอร์มีชื่อเดียวกับชื่อคลาส และ (2) *** ไม่มีประเภทการคืนค่าใดๆ (ไม่มี void)

คอนสตรัคเตอร์ (constructor)

```
public class Student{  
    private String id, name;  
    private double gpa;  
    ① public Student (){} // Default constructor คือ คอนสตรัคเตอร์ที่ไม่มีการรับค่า  
    ② public Student (String id, String name) {  
        this.id = id;  
        this.name = name;  
        this.gpa = 0.0;  
    }  
    ③ public Student (String id, String name, double gpa) {  
        this.id = id;  
        this.name = name;  
        this.gpa = gpa;  
    }  
}
```

กำหนด / ไม่กำหนดงานก็ได้

* ไม่บังคับ

overloaded

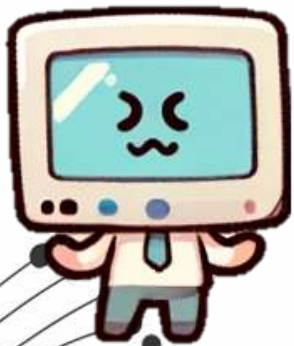
คอนสตรัคเตอร์ (constructor)

```
public class Student{
    private String id, name;
    private double gpa;
    public Student (){
        id = "unknown";
        name = "unknown";
        gpa = 0.0;
    } public Student (String id, String name) {
        this.id = id;
        this.name = name;
        this.gpa = 0.0;
    } public Student (String id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }
}
```

สำหรับตัวนี้

คอนสตรัคเตอร์ (constructor)

คอนสตรัคเตอร์จะถูกเรียกใช้งาน
อัตโนมัติเมื่อมีการสร้างอ็อบ
เจกต์จากคลาสนั้นๆ ผ่านคำสั่ง
new



```
public class Person{  
    private String name;  
    private double weight;  
    public Person (){}  
    public Person (String name){  
        this.name = name;  
    }  
    public Person (String name, double weight){  
        this.name = name;  
        this.weight = weight;  
    }  
    // setter and getter methods  
}
```

```
public class Main{  
    public static void main( String[] args) {  
        Person p1 = new Person();  
        Person p2 = new Person("Taravichet");  
        Person p3 = new Person("Amonthep", 75);  
        System.out.println("p1 "+ p1.getName() );  
        System.out.println("p2 "+ p2.getName() );  
        System.out.println("p3 "+ p3.getName() );  
    }  
}
```

4 constructor มีประเภทรุ่นกับ obj.
ไม่ได้นำประเภทรุ่นกับ class

ฟังก์ชัน constructor

คอนสตรัคเตอร์ (constructor)

```
public class Person{
    private String name;
    private double weight;
    // setter and getter methods
}
public class Main{
    public static void main( String[] args) {
        Person p1 = new Person();
        System.out.println("p1 "+ p1.getName() );
    }
}
```

Java แบบเต็ม

- ก) ทุก class ต้องมี class เอง
→ ต้อง extends obj.
- ข) ทุก class ต้องมี constructor อย่างน้อย
1 อัน ถ้าไม่มีให้
→ ต้อง default constructor มาให้

Default constructor จะถูกสร้างโดยอัตโนมัติในกรณีที่ คลาสนั้นไม่มีคอนสตรัคเตอร์อยู่เลย *

คอนสตรัคเตอร์ (constructor)

```
public class Person{
    private String name;
    private double weight;
    public Person (String name, double weight){
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p1 = new Person();
        System.out.println("p1 "+ p1.getName() );
    }
}
```

นี่คือ constructor ที่ไม่รับค่า

```
Main.java:11: error: constructor Person in cl
ass Person cannot be applied to given types;
    Person p1 = new Person();
                  ^
    required: String,double
    found:    no arguments
    reason: actual and formal argument lists di
ffer in length
```

หรือสามารถกล่าวได้ว่า ถ้าคลาสดังกล่าวมีการสร้างคอนสตรัคเตอร์อยู่แล้ว Default constructor จะไม่ถูกสร้างให้

หลักการทำงานของคอนสตรัคเตอร์กับการสร้างวัตถุ

ประกอบด้วย 4 ขั้นตอน

- จองพื้นที่หน่วยความจำให้กับวัตถุ
- กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของวัตถุ
- กำหนดค่าแอตทริบิวต์ของวัตถุตามที่ได้ประกาศไว้
- เรียกใช้งานคอนสตรัคเตอร์

```
public class Person{
    private String name = "unknown";
    private double weight = 1.0;

    public Person (String name, double weight) {
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p = new Person("Taravichet", 75);
    }
}
```

หลักการทำงานของคอนสตรัคเตอร์กับการสร้างวัตถุ

ประกอบด้วย 4 ขั้นตอน

- จองพื้นที่หน่วยความจำให้กับวัตถุ
- กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของวัตถุ
- กำหนดค่าแอตทริบิวต์ของวัตถุตามที่ได้ประกาศไว้
- เรียกใช้งานคอนสตรัคเตอร์

p  name 
Memory... weight 

```
public class Person{
    private String name = "unknown";
    private double weight = 1.0;

    public Person (String name, double weight) {
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p = new Person("Taravichet", 75);
    }
}
```


หลักการทำงานของคอนสตรัคเตอร์กับการสร้างวัตถุ

ประกอบด้วย 4 ขั้นตอน

- จองพื้นที่หน่วยความจำให้กับวัตถุ
- กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของวัตถุ → ตามระบบ
- กำหนดค่าแอตทริบิวต์ของวัตถุตามที่ได้ประกาศไว้
- เรียกใช้งานคอนสตรัคเตอร์

p  name null

Memory... weight 0.0

```
public class Person{
    private String name = "unknown";
    private double weight = 1.0;

    public Person (String name, double weight) {
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p = new Person("Taravichet", 75);
    }
}
```

หลักการทำงานของคอนสตรัคเตอร์กับการสร้างวัตถุ

ประกอบด้วย 4 ขั้นตอน

- จองพื้นที่หน่วยความจำให้กับวัตถุ
- กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของวัตถุ $\rightarrow \text{BizLog}\epsilon$
- กำหนดค่าแอตทริบิวต์ของวัตถุตามที่ได้ประกาศไว้ $\rightarrow \text{ตาม user ระบุ}$
- เรียกใช้งานคอนสตรัคเตอร์

p		name	unknown
Memory...		weight	1.0

```
public class Person{
    private String name = "unknown";
    private double weight = 1.0;

    public Person (String name, double weight) {
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p = new Person("Taravichet", 75);
    }
}
```

หลักการทำงานของคอนสตรัคเตอร์กับการสร้างวัตถุ

ประกอบด้วย 4 ขั้นตอน

- จองพื้นที่หน่วยความจำให้กับวัตถุ
- กำหนดค่าเริ่มต้นให้กับแอตทริบิวต์ของวัตถุ
- กำหนดค่าแอตทริบิวต์ของวัตถุตามที่ได้ประกาศไว้
- เรียกใช้งานคอนสตรัคเตอร์

p  name Taravichet
weight 75.0
Memory...

```
public class Person{
    private String name = "unknown";
    private double weight = 1.0;

    public Person (String name, double weight) {
        this.name = name;
        this.weight = weight;
    }
    // setter and getter methods
}

public class Main{
    public static void main( String[] args) {
        Person p = new Person("Taravichet", 75);
    }
}
```

หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด `this` และ `super` และเมธอด `this()` และ `super()`
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟส (Interface)



คีย์เวิร์ด this, super และเมธอด this(), super()

โดยปกติแล้วนักศึกษาสามารถใช้คีย์เวิร์ด this และ super เพื่อใช้ระบุว่าเป็นของคลาสแม่หรือคลาสลูกสำหรับกรณีที่มีความคลุมเครือ นอกจากนี้ ในภาษาจาวายังมี วิธีการเรียกคอนสตรัคเตอร์ระหว่างกันอยู่ โดยอาศัยเมธอด this(...) และ super(...) ซึ่งสามารถสรุปความแตกต่างได้ดังนี้

	ประเภท	บ่งบอกถึง	ความหมาย
this	คีย์เวิร์ด	คลาสตนเอง	ใช้เพื่อเรียกใช้งานแอททริบิวต์หรือเมธอด <u>ภายในคลาส</u> ซึ่งนิยมเรียกใช้งานเมื่อเกิดความคลุมเครือเท่านั้น และควรใช้กับแอททริบิวต์
super	คีย์เวิร์ด	คลาสแม่	ใช้เพื่อเรียกใช้งานเมธอดของ <u>คลาสแม่</u>
this(...)	เมธอด	คลาสตนเอง	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ <u>ภายในคลาส</u>
super(...)	เมธอด	คลาสแม่	เมธอดที่เรียกใช้งานเพื่อเรียกคอนสตรัคเตอร์ของ <u>คลาสแม่</u>

ตัวอย่างการใช้งานคีย์เวิร์ด this

นักศึกษาจะพบว่าเกิดความคลุมเครือของตัวแปร creditCardNumber และแอททริบิวต์ creditCardNumber ขึ้นในเมธอด setCreditCardNumber (..)

```
public class CreditCard{
    protected String creditCardNumber;
    protected int CCV;
    protected String cardHolderName;

    public CreditCard(){
    public void setCreditCardNumber(String creditCardNumber){
        this.creditCardNumber = creditCardNumber;
    }
    public void showCreditCardNumber(){
        System.out.print(creditCardNumber);
    }
}
```

คำนี้คือ attribute

ตัวอย่างการใช้งานคีย์เวิร์ด super

ในตัวอย่างนี้ super.display() ในคลาส Dog เรียกเมธอด display() จากซูเปอร์คลาส Animal เป็นวิธีที่มีประโยชน์ในการเรียกใช้เมธอดที่ถูกโอเวอร์ไรด์จากซูเปอร์คลาส***

```
public class Animal {
    public void display() {
        System.out.println("I am an animal");
    }
}
public class Dog extends Animal {
    public void display() {
        super.display(); // เรียกเมธอด display ของคลาส Animal
        System.out.println("I am a dog");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.display();
    }
}
```

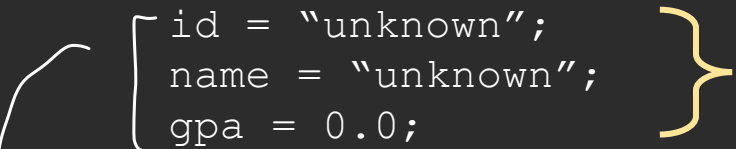
Output:
I am an animal
I am a dog

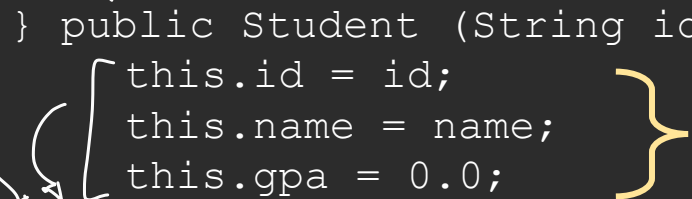
ตัวอย่างการใช้งานเมธอด this()

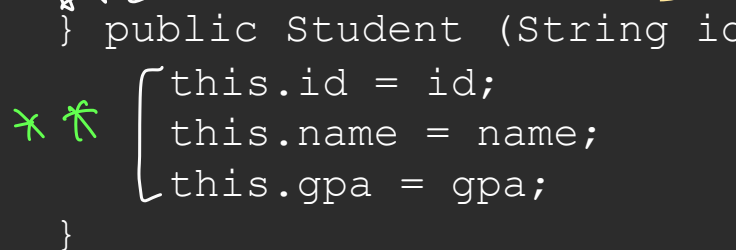
คือเวิร์ด this() ใช้เพื่อ เรียกคอนสตรัคเตอร์อื่นภายในคลาสเดียวกัน มันมีประโยชน์ในสถานการณ์ที่คลาสมีหลายคอนสตรัคเตอร์ที่มีการดำเนินการที่คล้ายกัน ใช้ this() เพื่อลดการซ้ำซ้อนของโค้ด โดยการเรียกคอนสตรัคเตอร์หนึ่งจากอีกคอนสตรัคเตอร์หนึ่ง

```
public class Student{
    protected String id, name;
    protected double gpa;
    public Student (){
        id = "unknown";
        name = "unknown";
        gpa = 0.0;
    }
    public Student (String id, String name) {
        this.id = id;
        this.name = name;
        this.gpa = 0.0;
    }
    public Student (String id, String name, double gpa) {
        this.id = id;
        this.name = name;
        this.gpa = gpa;
    }
}
```

this() → เรียกใช้ const. ของตัวเอง
ไปเรียกใช้ตามจำนวน parameter ที่ตรงกัน

 } this("unknown", "unknown", 0.0);

 } this(id, name, 0.0);

 } this(id, name, gpa);

ตัวอย่างการใช้งานเมธอด super()

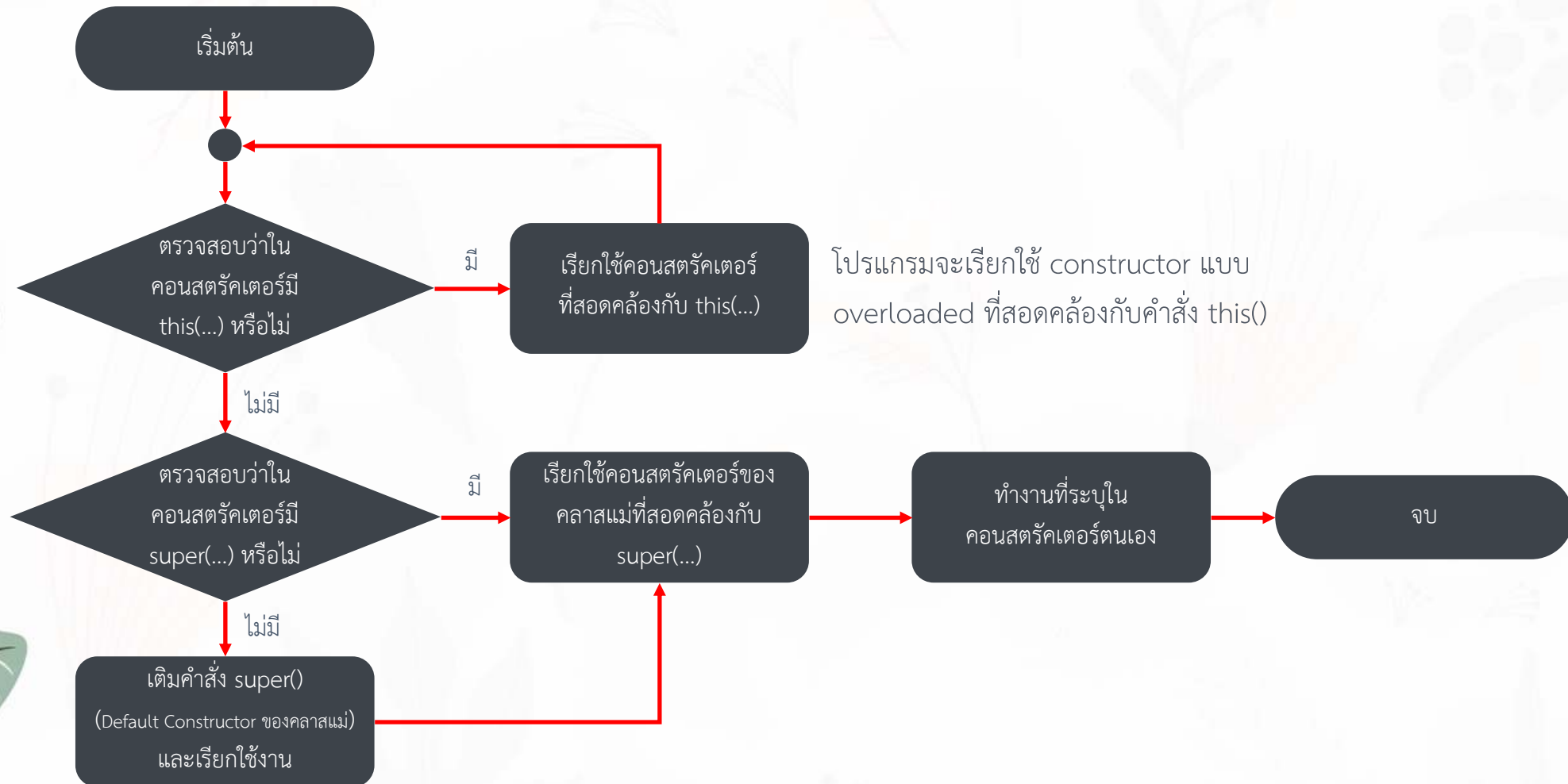
คีย์เวิร์ด super() ใช้เพื่อเรียกคอนสตรักเตอร์ของคลาสแม่จากคลาสลูก การใช้ super() มีประโยชน์เมื่อต้องการให้คลาสลูกเรียกคอนสตรักเตอร์ของคลาสแม่เพื่อทำการตั้งค่าเริ่มต้นหรือดำเนินการที่จำเป็นก่อนที่จะดำเนินการเพิ่มเติมในคลาสลูก

```
public class BankAccount {  
    protected double balance;  
    public BankAccount(double balance) {  
        if (balance > 0) {  
            this.balance = balance;  
        }  
    }  
}  
  
public class SavingsAccount extends BankAccount {  
    private double interestRate;  
    public SavingsAccount(double balance, double interestRate) {  
        super(balance); # เรียกใช้ constructor ของแม่ที่ กำหนดไว้ค่าเท่านั้น  
        this.interestRate = interestRate;  
    }  
}
```

ตัวอย่างการใช้งานเมธอด super()

```
public class Animal{
    protected double weight;
    public Animal (double weight) {
        this.weight = weight;
    }
}
public class Pig extends Animal{
    protected double height;
    public Pig (){
        super(0);
        this.height = 0; } หรือ this(0,0);
    } public Pig (double height, double weight) {
        super(weight);
        this.height = height;
    }
}
```

ขั้นตอนการทำงานของคอนสตรัคเตอร์



โปรแกรมจะเรียกใช้ constructor แบบ default ของ superclass ยกเว้นคลาส object เพราะไม่มี superclass

ตัวอย่างการทำงานของคอนสตรัคเตอร์

```
public class แม่Animal {  
    con→ public Animal() {  
        System.out.println("Hi, Animal");  
    }  
}  
  
public class ลูกAnt extends Animal {  
    con→ public Ant() { super();  
        System.out.println("Hi, Ant");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Ant a1 = new Ant();  
    }  
}
```

javac แอมบ์พิมพ์

③ ใน class ของลูก ถ้าไม่มีเขียนไว้ class ของแม่
ถ้าไม่มีใส่ javac จะพิมพ์ในน้อง
↳ super()

สั่งเรียก class ของแม่เป็นอันดับแรกเสมอ

ผลลัพธ์

Hi, Animal

Hi, Ant

ตัวอย่างการทำงานของคอนสตรัคเตอร์

```
public class Animal {  
    public Animal(){  
        System.out.println("Hi, Animal");  
    }  
}
```

```
public class Ant extends Animal {  
    public Ant(){  
        this("Ant");  
    }  
    public Ant(String n){  
        System.out.println("Hi, " + n);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Ant a1 = new Ant("Aaaa");  
    }  
}
```

(2)

(3)

ต้องเรียก constructor ของ父 class ก่อน
จึงจะเรียก constructor ของตัวเองได้

ผลลัพธ์

Hi, Animal

Hi, Aaaa

ข้อควรระวังเกี่ยวกับ super()

จากโค้ดจะพบข้อผิดพลาดเมื่อพยายามสร้างอินสแตนซ์ของคลาส Duck ด้วย คอนสตรักเตอร์ Duck() (ที่ไม่มีอาร์กิวเมนต์) เนื่องจากคลาส Duck ได้สืบทอดมาจากคลาส Animal ซึ่งมีคอนสตรักเตอร์ที่ต้องการอาร์กิวเมนต์ String แต่คลาสลูกจำเป็นต้องเรียกคอนสตรักเตอร์ของคลาสแม่ผ่าน super() ทุกครั้งเป็นคำสั่งแรกในทุกคอนสตรักเตอร์ สิ่งนี้เป็นข้อกำหนดในภาษาจาวา ในกรณีนี้ คอนสตรักเตอร์ Duck() (ที่ไม่มีอาร์กิวเมนต์) ไม่ได้เรียก super() ที่สอดคล้องกับที่ปรากฏในคลาส Animal จึงเป็นสาเหตุของข้อผิดพลาด

```
public class Main {  
    public static void main(String[] args) {  
        Duck d2 = new Duck();  
    }  
}
```

```
public class Animal{  
    protected String name;  
    public Animal(String n){  
        this.name = n;  
    }  
}
```

```
public class Duck extends Animal{  
    public Duck(String n){  
        super(n);  
    }  
    public Duck(){  
        // super(); # ขาดการเรียก  
    }  
}
```

นี่คือ default const.

ลองลบเพิ่มไปเลย error

ผลลัพธ์

Main.java:14: error: constructor Animal in class Animal cannot be applied to given types;

```
    public Duck(){  
        ^
```

required: String

found: no arguments

วิธีการแก้ไขที่ 1 กำหนด super() ด้วยตนเอง

```
public class Main {  
    public static void main(String[] args) {  
        Duck d1 = new Duck("Pedd");  
        Duck d2 = new Duck();  
    }  
}  
  
public class Animal{  
    protected String name;  
    public Animal(String n){ this.name = n; }  
}  
  
public class Duck extends Animal{  
    public Duck(String n){  
        super(n);  
    }  
    public Duck(){  
        super("");  
    }  
}
```

เพิ่มเองให้มันไม่ผิดพลาด

วิธีการแก้ไขที่ 2 เพิ่ม Default Constructor

```
public class Main {  
    public static void main(String[] args) {  
        Duck d = new Duck();  
        System.out.println("d's name is "+ d.getName());  
    }  
}  
  
public class Animal{  
    protected String name;  
    public Animal(){  
        this("unknown");  
    } public Animal(String n){  
        setName(n);    // ควรกำหนดค่าผ่าน setter  
    }  
    public String getName(){ return this.name; }  
    public void setName(String name){ this.name = name; }  
}  
  
public class Duck extends Animal{  
    public Duck(String n){  
        super(n);  
    } public Duck(){  
  
    }  
}
```

ไม่ต้องเพิ่ม default constructor ให้เพิ่ม explicit 1 ตัว

ผลลัพธ์

d2's name is unknown

คีย์เวิร์ด final

คีย์เวิร์ด final สามารถใช้ได้กับคลาส ตัวแปร แอททริบิวต์ และเมธอด

ตำแหน่ง	ความหมาย
คลาส	ทำให้คลาสอื่นไม่สามารถสืบทอดคลาสนี้ได้
เมธอด	เมธอดไม่สามารถ overridden ได้
ตัวแปร	ค่าคงที่ ซึ่งจะทำให้สามารถกำหนดค่าได้เพียงครั้งเดียวเท่านั้น

คีย์เวิร์ด final กับคลาส (final class)

เมื่อใช้ `final` กับคลาสจะหมายความว่าคลาสนั้นไม่สามารถถูกสืบทอด (extend) ได้ นี่คือการป้องกันไม่ให้คลาสอื่นขยายคลาส `final`

```
final class Constants {  
    public static final double PI = 3.14159;  
}  
  
// คลาสนี้ไม่สามารถถูกสืบทอดได้เนื่องจาก Constants เป็น final  
public class MoreConstants extends Constants { // ข้อผิดพลาด!  
    // ...  
}
```

คีย์เวิร์ด final กับตัวแปร (final variable)

เมื่อใช้ final กับตัวแปรๆ นั้นจะไม่สามารถมีการเปลี่ยนแปลงค่าหลังจากที่ถูกกำหนดค่าครั้งแรกได้ ตัวแปร final มักใช้สำหรับค่าคงที่

```
public class Circle {  
    public final double radius;  
  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
  
    public static void main(String[] args) {  
        Circle c = new Circle(5.0);  
        c.radius = 10.0; // ข้อผิดพลาด! ไม่สามารถมีการเปลี่ยนแปลงค่า radius เนื่องจากเป็น final  
    }  
}
```

คีย์เวิร์ด **final** กับเมธอด (final method)

```
public class BankAccount {
    private double balance;
    public BankAccount(double initialBalance) { this.balance = initialBalance; }
    public final boolean withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("จำนวนเงินที่จะถอนต้องมากกว่าศูนย์");
            return false;
        }
        if (balance >= amount) {
            balance -= amount;
            System.out.println("ถอนเงินจำนวน " + amount + " บาท");
            return true;
        } else {
            System.out.println("ยอดเงินไม่พอสำหรับการถอน");
            return false;
        }
    }
}
```

เมธอดที่ถูกประกาศเป็น **final** จะไม่สามารถถูกโอเวอร์ไรด์ (overridden) ในคลาสลูกได้ นี่เป็นวิธีที่ช่วยให้สามารถป้องกันพฤติกรรมของเมธอดในการสืบทอด

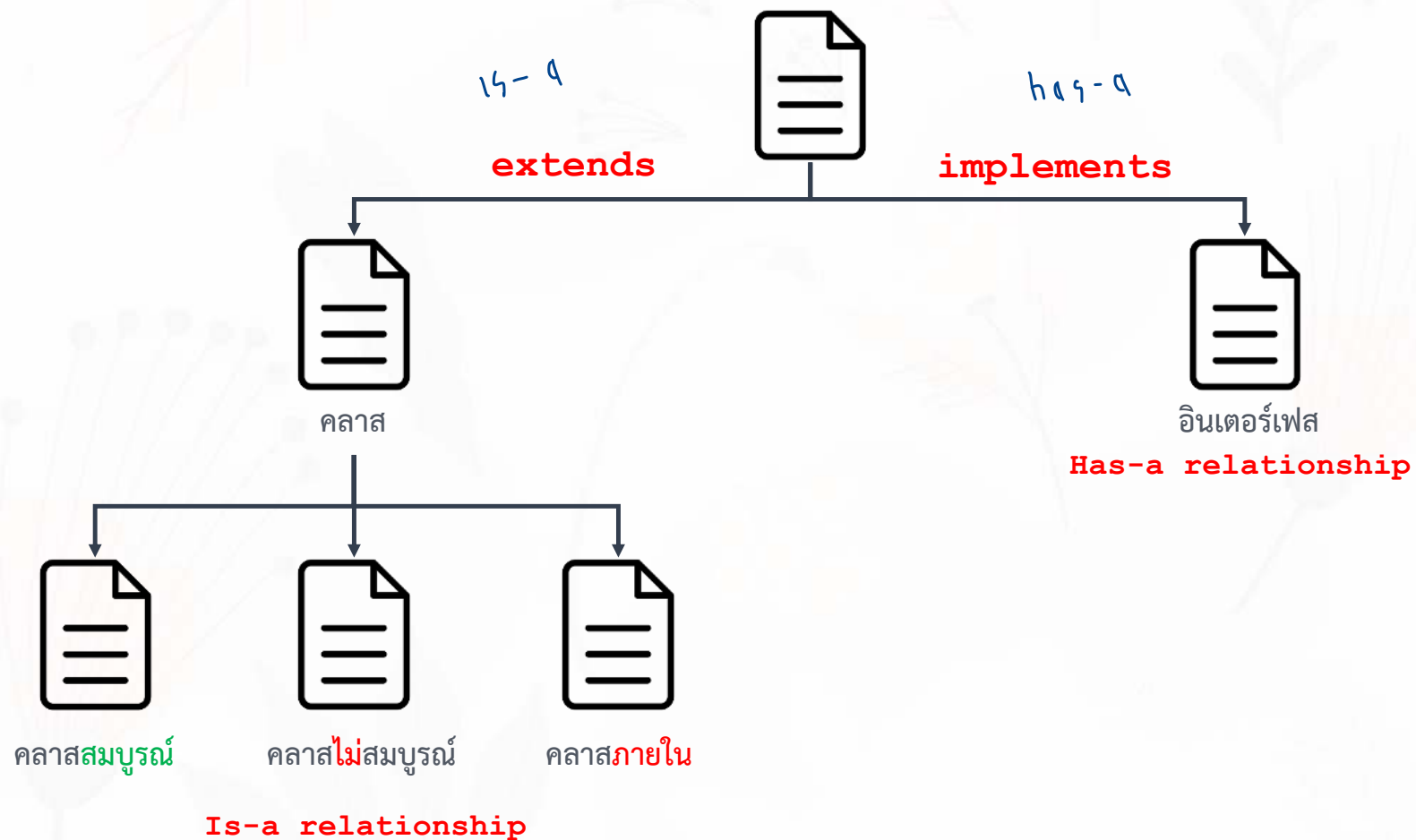
```
public class SavingsAccount extends BankAccount {
    private double interestRate;
    public SavingsAccount(double initialBalance, double interestRate) {
        super(initialBalance);
        this.interestRate = interestRate;
    }
    // ไม่สามารถโอเวอร์ไรด์เมธอดถอนเงินได้เนื่องจากเป็น final
    // public boolean withdraw(double amount) {
    //     balance -= amount;
    //     System.out.println("ถอนเงินจำนวน " + amount + " บาท");
    // }
}
```


หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - ~~คลาสภายใน (Inner class หรือ Nested class) ไม่ผ่าน~~
- อินเตอร์เฟส (Interface)



การสืบทอดโดยอ้างอิงจากความสัมพันธ์



หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟส (Interface)



คลาสไม่สมบูรณ์ (Abstract Class)

↳ คลาสที่สร้าง obj จากมันไม่ได้ ฝ่ไว้เพื่อให้สืบทอดไปใช้

Vehicle *	
- fuel	: int
- topSpeed	: String
# setFuel(int i)	: void
# setTopSpeed(String n)	: void
# getFuel()	: int
# getTopSpeed()	: String
+ showInfo()	: void
+ move()	: void

Car	
- typeEngine	: String
+ setTypeEngine(String t)	: void
+ getTypeEngine()	: String
+ showCarInfo()	: void
+ setCarInfo(int s, String t, String y)	: void

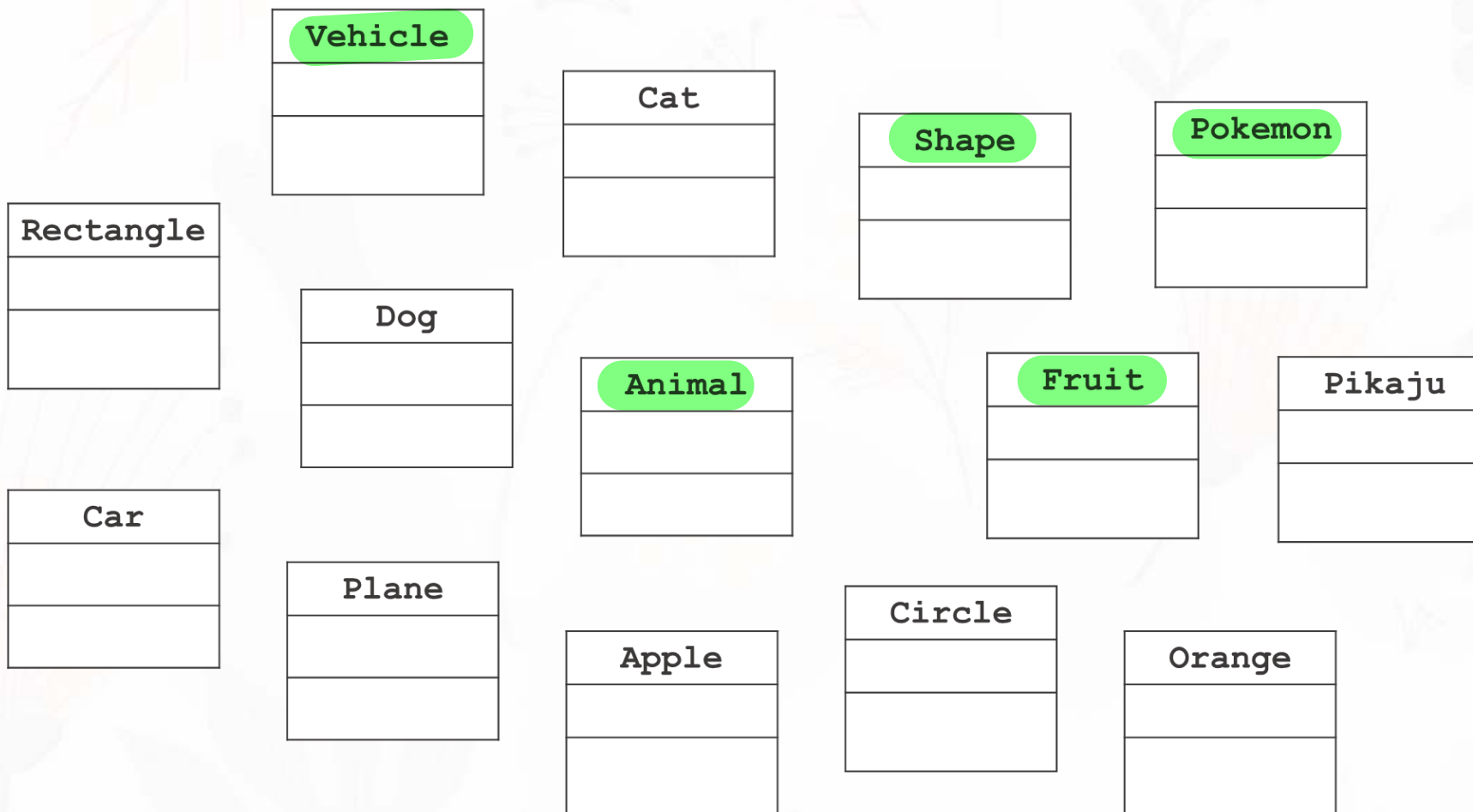
Plane	
+ showPlaneInfo()	: void
+ setPlaneInfo(int s, String t)	: void

คลาสไม่สมบูรณ์ คือ คลาสที่ไม่สามารถสร้างเป็นอินสแตนซ์ได้โดยตรง ซึ่งมักถูกใช้เป็นคลาสพื้นฐานในการสืบทอด (inheritance) และมีการกำหนดเมธอดที่สมบูรณ์และไม่สมบูรณ์ด้วยคีย์ **abstract** คลาสลูกที่สืบทอดจากคลาสนี้จะต้องทำการโอเวอร์ไรด์เมธอดเหล่านั้น โดยคลาสไม่สมบูรณ์สามารถมี

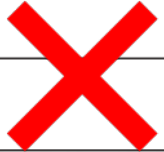
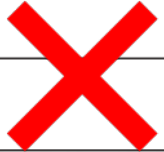
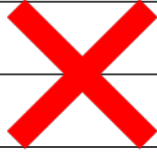
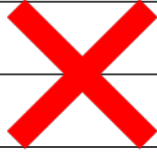
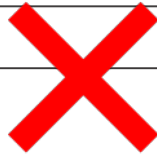
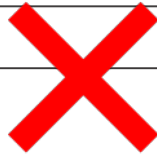
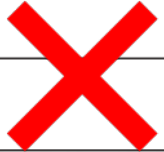
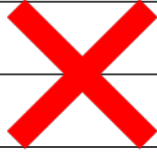
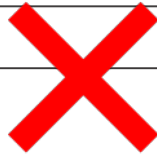
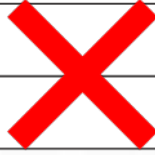
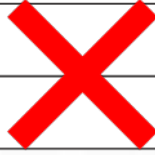
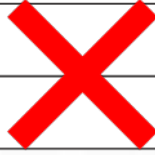
- เมธอดไม่สมบูรณ์ (abstract methods) ซึ่งเป็นเพียงการประกาศเมธอดโดยไม่มีการกำหนดโค้ดบล็อก คลาสลูกจะต้องกำหนดการใช้งานเมธอดเหล่านั้น
- เมธอดที่สมบูรณ์ (non-abstract methods) ซึ่งเมธอดเหล่านี้มีการกำหนดโค้ดบล็อกแล้ว คลาสลูกจะโอเวอร์ไรด์หรือไม่ก็ได้

คลาสไม่สมบูรณ์ (Abstract Class)

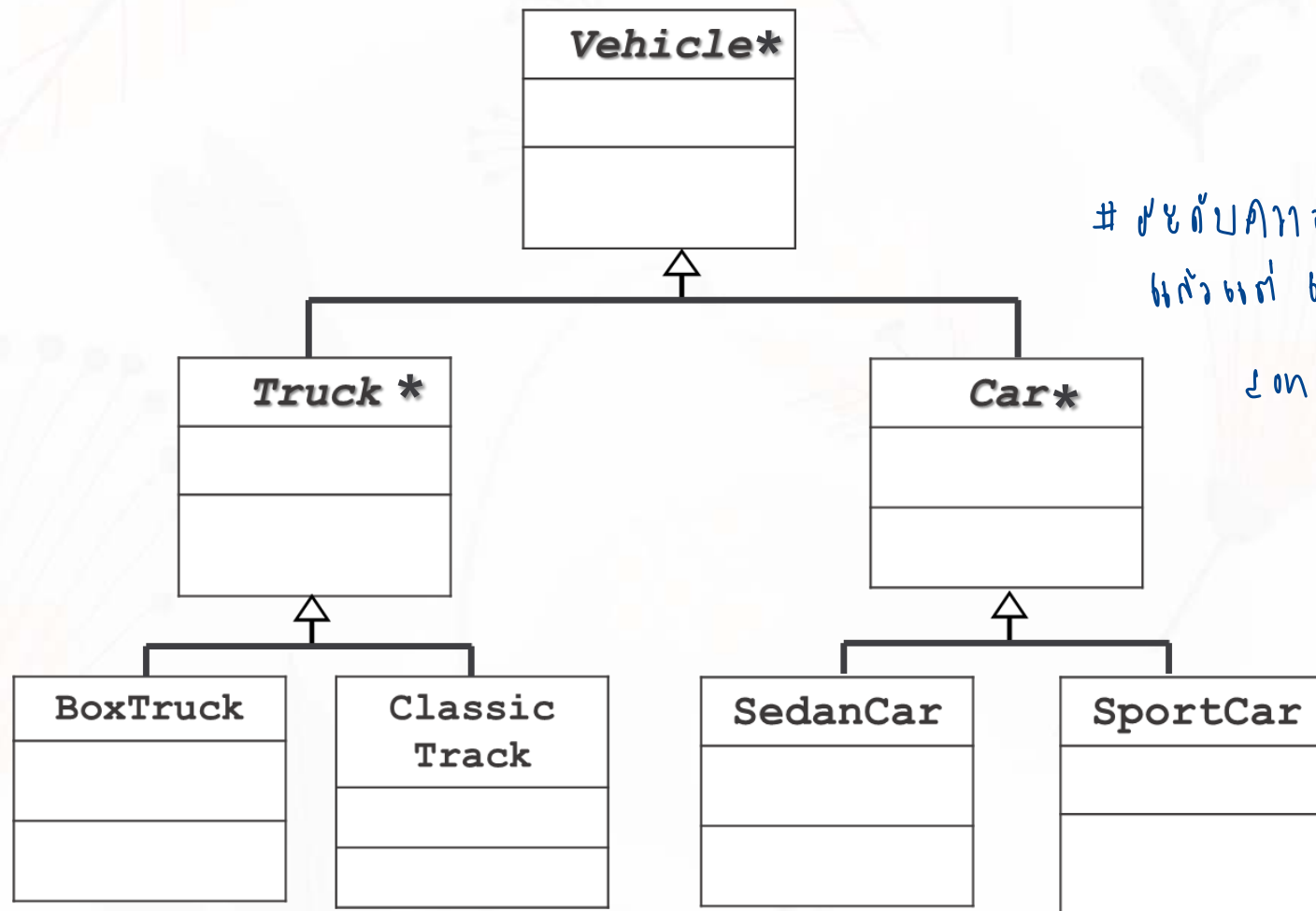
● → 6 เป็น abstract



คลาสไม่สมบูรณ์ (Abstract Class)

	<table><tr><td>Vehicle</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Vehicle			<table><tr><td>Cat</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Cat			<table><tr><td>Shape</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Shape			<table><tr><td>Pokemon</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Pokemon					
Vehicle																			
																			
Cat																			
																			
Shape																			
																			
Pokemon																			
																			
<table><tr><td>Rectangle</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Rectangle			<table><tr><td>Dog</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Dog			<table><tr><td>Animal</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Animal			<table><tr><td>Fruit</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Fruit			<table><tr><td>Pikaju</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Pikaju		
Rectangle																			
																			
Dog																			
																			
Animal																			
																			
Fruit																			
																			
Pikaju																			
																			
<table><tr><td>Car</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Car			<table><tr><td>Plane</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Plane			<table><tr><td>Apple</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Apple			<table><tr><td>Circle</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Circle			<table><tr><td>Orange</td></tr><tr><td></td></tr><tr><td></td></tr></table>	Orange		
Car																			
																			
Plane																			
																			
Apple																			
																			
Circle																			
																			
Orange																			
																			

คลาสไม่สมบูรณ์ (Abstract Class)



ใช้กับภาพ abstract
สำหรับ biz logic,
condition

คลาสไม่สมบูรณ์ (Abstract Class)

คือ คลาสนั้นมีเมธอดแบบไม่สมบูรณ์อย่างน้อยหนึ่งเมธอดอยู่ในคลาส ในทางปฏิบัติอาจจะไม่มีเมธอดที่ยังไม่สมบูรณ์เลยก็ได้ โดยอาศัย คีย์ abstract เป็นตัวกำหนด

ห้ไม่สมบูรณ์
ตั้งแต่ () ไม่ให้ { } body

รูปแบบของเมธอดที่ไม่สมบูรณ์

```
[modifier] abstract return_type methodName([arguments]);
```

คลาสไม่สมบูรณ์กำหนดขึ้นมาเพื่อให้คลาสอื่นสืบทอด โดยคลาสที่มาสืบทอดจะต้องกำหนดโค้ดบล็อกคำสั่งในเมธอดที่ยังไม่สมบูรณ์ (บังคับ) นอกจากนี้ ยังไม่สามารถสร้างอ็อบเจกต์ของคลาสแบบ abstract ได้

ตัวอย่างโปรแกรมคลาสไม่สมบูรณ์ที่ 1

```
public abstract class Shape {  
    public Shape() {}  
  
    // Abstract method  
    public abstract double getArea();  
}  
  
public class Circle extends Shape {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    @Override  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // ไม่สามารถสร้างอินสแตนซ์ของคลาส Shape ได้โดยตรงเพราะเป็นคลาสไม่สมบูรณ์  
        Shape s = new Shape("Red"); // ข้อผิดพลาด!  
    }  
}
```

ไม่ใส่ {}
method ไม่สมบูรณ์
ใส่ ; ปิดแทน

1. ลืม

2. ใส่ abstract ที่ class, method ที่

3. ปิดวงเล็บ

4. ปิด ;

Main.java:4: error: Shape is abstract; cannot be instantiated

Shape s = new Shape("Red");

ตัวอย่างโปรแกรมคลาสไม่สมบูรณ์ที่ 2

```
public abstract class Shape {  
    public Shape() {}  
  
    // Abstract method  
    public abstract double getArea();  
}  
  
public class Circle extends Shape {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    @Override  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        // สามารถสร้างอินสแตนซ์ของคลาส Circle ซึ่งสืบทอดจาก Shape  
        Circle c = new Circle(2.5);  
        System.out.println(c.getArea()); // แสดงผลพื้นที่ของวงกลม  
    }  
}
```

ตัวอย่างโปรแกรมคลาสไม่สมบูรณ์ที่ 3

```
public abstract class Shape {  
    public Shape() {}  
  
    // Abstract method  
    public abstract double getArea();  
}  
  
public class Circle extends Shape {  
    private double radius;  
    public Circle(double radius) {  
        this.radius = radius;  
    }  
    @Override  
    public double getArea() {  
        return Math.PI * radius * radius;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Shape s = new Circle(2.5);  
        System.out.println(s.getArea());  
    }  
}
```

วงกลมที่ไม่มีตัวแปรรัศมี

???

สถานการณ์ที่ควรใช้หลักการ **คลาสไม่สมบูรณ์**

การใช้คลาสไม่สมบูรณ์เป็นกลยุทธ์การออกแบบที่สำคัญในการโปรแกรมแบบวัตถุ เพราะช่วยในการกำหนดแนวทางและโครงสร้างพื้นฐานให้กับคลาสลูก ขณะเดียวกันก็ยังให้ความยืดหยุ่นในการขยายและปรับเปลี่ยนโค้ดได้

เพื่อไม่ให้ตัวเองเปลี่ยนงงใน abstract class

- เมื่อ **คลาสมีเมธอดที่คลาสลูกทุกตัวควรมีแต่การทำงานอาจต่างกัน** เช่น คลาส Shape ที่มีเมธอด `getArea()` แต่การคำนวณพื้นที่ขึ้นอยู่กับชนิดของรูปทรง
- เมื่อต้องการ **จัดกลุ่มคลาสที่มีคุณสมบัติทั่วไป** เช่น คลาส Animal ที่มีคุณสมบัติทั่วไปเช่น `age` และ `weight` และเมธอด `move()` ที่ทุกสัตว์จะต้องเคลื่อนที่ แต่ละชนิดอาจจะเคลื่อนที่ไม่เหมือนกัน
- เมื่อ คลาสพื้นฐาน **ไม่ควรสร้างเป็นอินสแตนซ์** บางครั้งคลาสพื้นฐานอาจไม่มีความหมายเมื่อสร้างเป็นอินสแตนซ์โดยตรง เช่น คลาส Animal ที่ควรมีเพียงคลาสลูกเช่น Bird ที่สามารถสร้างเป็นอินสแตนซ์ได้
- เมื่อต้องการ **ควบคุมการออกแบบและโครงสร้างของคลาสลูก** เพื่อให้แน่ใจว่าคลาสลูกทุกตัวจะปฏิบัติตามแบบแผนที่กำหนดไว้ในคลาสไม่สมบูรณ์

หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟซ (Interface)



อินเทอร์เฟส (Interface)

เน้นร่วม method ที่ไม่สมบูรณ์

ไม่มี body

คือ โครงสร้างที่ใช้ในการกำหนดชุดของเมธอดแบบไม่สมบูรณ์ ที่ไม่มีการระบุการทำงาน (implementation) ไว้เลย คลาสที่ "implements" อินเทอร์เฟสนั้นจะต้องกำหนดการทำงานของเมธอดเหล่านั้นทั้งหมด อินเทอร์เฟสให้ความสามารถในการกำหนดชนิดของการกระทำที่คลาสควรจะทำโดยไม่จำเป็นต้องกำหนดวิธีการทำงานเหล่านั้น อินเทอร์เฟสเป็นหนึ่งในวิธีการใช้หลักการของการโปรแกรมแบบวัตถุที่ชื่อว่า "การกำหนดรูปแบบการทำงาน" (Abstraction) และ "การกำหนดสัญญา" (Contract) สำหรับคลาสที่จะใช้งานอินเทอร์เฟส โดยมีรูปแบบดังนี้

๒ ตัวส่วที่ทำงานจับคู่กัน

```
[modifier] interface InterfaceName {  
    [methods() ;]
```

```
}
```

๑ ประกาศเมื่อก่อน class

อินเทอร์เฟสกำหนดขึ้นมาเพื่อให้คลาสอื่นนำไปใช้งาน โดยใช้คีย์เวิร์ด implements โดยมีรูปแบบดังนี้

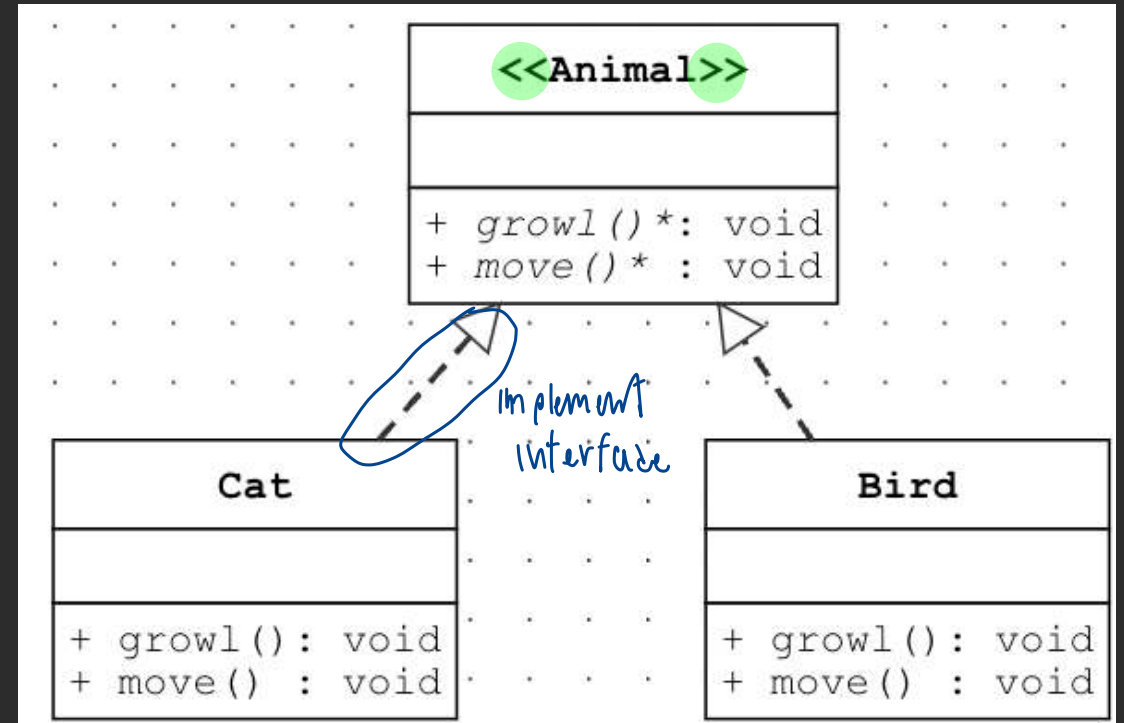
```
[modifier] class ClassName implements InterfaceName {  
    [methods() ;]  
}
```

ลักษณะของอินเทอร์เฟส

- ไม่มีสถานะ คือ อินเทอร์เฟสไม่สามารถมีแอตทริบิวต์ (ไม่สามารถเก็บข้อมูลหรือสถานะได้) แต่ใน Java 8 ขึ้นไป, อินเทอร์เฟสสามารถมีแอตทริบิวต์ของคลาสได้ (static attribute) ที่มีค่าคงที่ได้ (final)
- การทำงานเปล่า คือ โดยทั่วไปอินเทอร์เฟสจะมีแต่เมธอดที่ไม่สมบูรณ์ ซึ่งเมธอดเหล่านี้จะต้องถูกโอเวอร์ไรด์ในคลาสที่ implements อินเทอร์เฟสนั้น
- ความเป็นหลายรูปแบบ (Polymorphism) คือ คลาสสามารถ implements ได้หลายอินเทอร์เฟส
- ~~เมธอดเริ่มต้น (Default Methods) คือ ตั้งแต่ Java 8 อินเทอร์เฟสสามารถมีเมธอดสมบูรณ์ได้แล้ว (Default Methods) ที่ไม่จำเป็นต้องโอเวอร์ไรด์ในคลาสที่ implements (ไม่แนะนำ)~~

ตัวอย่างอินเทอร์เฟส (Interface)

```
public interface Animal {  
    public abstract void growl();  
    public abstract void move();  
}  
  
public class Cat implements Animal {  
    @Override  
    public void growl() {  
        System.out.println("Mew Mew");  
    }  
    @Override  
    public void move() {  
        System.out.println("Walk");  
    }  
}  
  
public class Bird implements Animal {  
    @Override  
    public void growl() {  
        System.out.println("Jib Jib");  
    }  
    @Override  
    public void move() {  
        System.out.println("Fly");  
    }  
}
```



อินเทอร์เฟส (Interface)

เมื่อประกาศอินเทอร์เฟสดังนี้

```
public interface Moveable {  
    int AVERAGE_SPEED = 40;  
    void move();  
}
```

จาวาจะเพิ่มรายละเอียดของแอททริบิวต์และเมธอดของอินเทอร์เฟส ดังนี้ (1) แอททริบิวต์ของอินเทอร์เฟสจะกำหนดให้เป็น public static final และ (2) เมธอดของอินเทอร์เฟสจะกำหนดให้เป็น public abstract

```
public interface Moveable {  
    attribute public static final int AVERAGE_SPEED = 40;  
    method public abstract void move();  
}
```

⌚ #๖๖๖๖๖๖๖ Ⓢ ถ้าไม่ใส่จะเริ่ดเริ่ด: เริ่ดเริ่ดเริ่ดเริ่ด

อินเทอร์เฟส (Interface)

ในทางปฏิบัติแล้ว ถ้าผู้พัฒนาอยากได้แอททริบิวต์ของอินเทอร์เฟสที่ไม่ใช่ public static final อาจจะสามารถหลีกเลี่ยงได้ดังนี้

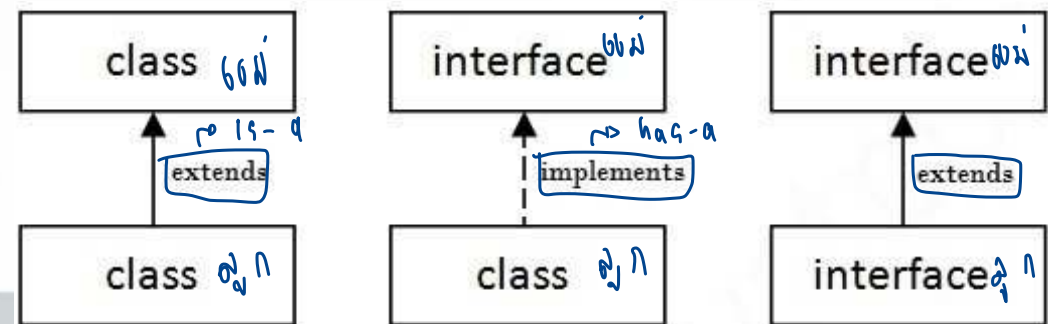
```
public interface Walkable {  
  
    //private double distance;  
  
    public abstract double getDistance();  
  
    public abstract void setDistance(double distance);  
  
}
```

ข้อกำหนดของอินเทอร์เฟซ (Interface)

- ทุกเมธอดภายในอินเทอร์เฟซไม่สามารถกำหนดให้เป็น static และ final ได้
- ทุกเมธอดภายในอินเทอร์เฟซจะถูกบังคับให้เป็น public abstract โดยอัตโนมัติ ถึงแม้ว่าจะไม่ได้กำหนด
- ทุกแอททริบิวต์ภายในอินเทอร์เฟซจะถูกกำหนดให้เป็น public static final หรือเป็นแอททริบิวต์ของคลาสที่เป็นค่าคงที่
- อินเทอร์เฟซ กับ อินเทอร์เฟซ จะ extends และสามารถ extends ได้มากกว่า 1 อินเทอร์เฟซ เช่น

```
public interface Hockey extends Sports, Event {  
  
}
```

- อินเทอร์เฟซไม่สามารถ implement คลาสได้
- อินเทอร์เฟซสามารถซ้อนกันได้



ตัวอย่างอินเทอร์เฟซที่ 1

```
public interface PaymentProcessor {  
    public abstract void processPayment(double amount);  
}  
  
public class CreditCardProcessor implements PaymentProcessor {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing credit card payment of $" + amount);  
    }  
}  
  
public class PayPalProcessor implements PaymentProcessor {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing PayPal payment of $" + amount);  
    }  
}
```


ตัวอย่างอินเทอร์เฟซที่ 2

```
public interface Runnable {  
    public static final int x = 10;  
    public abstract void move(int n);  
    public abstract void move();  
}
```

attribute

```
public interface Flyable {  
    public static final int x = 100;  
    public abstract void move();  
}
```

```
public class Pegasus implements Runnable, Flyable {  
    public static void main(String[] args) {
```

```
        // System.out.println(x); >>> error: reference to x is ambiguous
```

```
        System.out.println(Runnable.x);
```

```
        System.out.println(Flyable.x);
```

```
        new Pegasus().move();
```

```
    public void move() {
```

```
        move(1);
```

```
    public void move(int i) {
```

```
        for (int j = 1; j <= i; j++)
```

```
            System.out.println("M m M m");
```

```
    }
```

```
}
```

ข พหุ - ๓ - ๗ ๗.๗๗๗๗

ข ๗๗๗๗ ๗๗๗๗ interface ๗๗๗๗ interface.attribute

ผลลัพธ์

10

100

M m M m

ตัวอย่างอินเทอร์เฟซที่ 3

```
public interface Monster {  
    public void menace();  
}  
  
public interface DangerousMonster extends Monster {  
    public void destroy();  
}  
  
public interface Lethal {  
    public void kill();  
}  
  
public interface Vampire extends DangerousMonster, Lethal {  
    public void drinkBlood();  
}  
  
public class DragonZilla implements DangerousMonster {  
    public void menace() {}  
    public void destroy() {}  
}
```

ความแตกต่างกันของ Interface และ Abstract Class

	Interface ไม่สมบูรณ์ทุกประการ	Abstract Class
เมธอด	เมธอดทั้งหมดในอินเตอร์เฟสโดยปกติเป็นเมธอดไม่สมบูรณ์ ตั้งแต่ Java 8 เป็นต้นไป, อินเตอร์เฟสสามารถมี default method และ static method ที่มีการกำหนดการทำงานได้	สามารถมีทั้งเมธอดไม่สมบูรณ์และเมธอดสมบูรณ์
แอททริบิวต์	มีได้แต่แอททริบิวต์ของคลาสที่มีค่าคงที่	สามารถมีแอททริบิวต์ที่ไม่ใช่ค่าคงที่และแอททริบิวต์ปกติได้
การสืบทอด	คลาสสามารถ implements ได้หลายอินเตอร์เฟส	คลาสสามารถสืบทอดได้เพียงคลาสเดียวเท่านั้น
การใช้งาน	ใช้เมื่อคุณต้องการให้คลาสต่างๆ ที่ไม่มีความเกี่ยวข้องกันมีเมธอดที่เหมือนกัน	ใช้เมื่อคลาสต่างๆ ที่สืบทอดมามีความเกี่ยวข้องกันและแชร์โค้ดหรือสถานะตัวแปรในรูปแบบที่เหมือนกัน
การสร้างอินสแตนซ์	ไม่สามารถสร้างอินสแตนซ์จากอินเตอร์เฟสได้	ไม่สามารถสร้างอินสแตนซ์โดยตรงจากคลาสไม่สมบูรณ์ได้

การเลือกใช้งานระหว่าง Interface และ Abstract Class

โดยทั่วไป อินเทอร์เน็ตให้ความยืดหยุ่นมากกว่าและเป็นทางเลือกที่ดีเมื่อคุณต้องการที่จะกำหนดพฤติกรรมที่คลาสต่างๆ ควรมีโดยไม่ต้องกำหนดโครงสร้างหรือสถานะของคลาสเหล่านั้น

- Interface → has-a

นิยมใช้งานเมื่อคุณต้องการกำหนดสัญญา (contract) แบบเดียวกันให้กับคลาสต่างๆ ที่อาจไม่มีความสัมพันธ์กัน

- Abstract Class → is-a

นิยมใช้งานเมื่อคุณกำลังออกแบบคลาสที่มีความสัมพันธ์กันและมีโค้ดหรือสถานะตัวแปรที่เหมือนกัน

การประยุกต์ใช้คลาสไม่สมบูรณ์และอินเตอร์เฟส

```
public interface Flyable {  
    public abstract void fly();  
}
```

19 - 1 → ลีน่า นีน่า ลีน่า

happy ๕ → ป. ส. ๑๖/๖๖

```
public abstract class Animal {  
    public abstract void move();  
}
```

• → ลีน่า นีน่า ลีน่า

```
public class Plane implements Flyable {  
    public void fly() { System.out.println("Fly by Engine"); }  
}
```

```
public class Bird extends Animal implements Flyable {  
    public void fly() { System.out.println("Fly by Wing"); }  
    public void move() { System.out.println("Walk"); }  
}
```

```
public class Fish extends Animal {  
    public void move() { System.out.println("Swimming"); }  
}
```

การประยุกต์ใช้คลาสไม่สมบูรณ์และอินเทอร์เฟส

ข้อ: 78 63 ที่ 2144 นี้ให้ implement flyable

```
public class Main {  
    public void orderToFly(Flyable f) { f.fly(); }  
    public void orderToMove(Animal a) { a.move(); }  
    public Flyable returnFlyableObject(Flyable f) { return f; }  
    public Animal returnAnimal(Animal a) { return a; }  
  
    public static void main(String[] args) {  
        Bird b1 = new Bird();  
        Fish f1 = new Fish();  
        Plane p1 = new Plane();  
        Main m = new Main();  
        System.out.println("==== Case (I) =====");  
        m.orderToFly(p1);  
        m.orderToFly(b1);  
        m.orderToMove(f1);  
        m.orderToMove(b1);  
        System.out.println("==== Case (II) =====");  
        System.out.println(m.returnAnimal(f1));  
        System.out.println(m.returnAnimal(b1));  
        System.out.println(m.returnFlyableObject(p1));  
        System.out.println(m.returnFlyableObject(b1));  
    }  
}
```

ผลลัพธ์

run:

==== Case (I) =====

Fly by Engine

Fly by Wing

Swimming

Walk

==== Case (II) =====

Fish@214c265e

Bird@448139f0

Plane@7cca494b

Bird@448139f0

ชนิดของอินเทอร์เฟส

- **Standard Interface** คือ อินเทอร์เฟสที่มีเมธอดไม่สมบูรณ์และอาจจะมีแอททริบิวต์ของคลาสแบบค่าคงที่ มีรูปแบบดังนี้

```
public interface Queue{  
    public abstract boolean add(Object o);  
    public abstract Object get(int i);  
    public abstract boolean remove(int);  
}
```

- **Functional Interface** คือ อินเทอร์เฟสที่มีเมธอดไม่สมบูรณ์เพียงอันเดียวและไม่มีแอททริบิวต์ โดยมีรูปแบบดังนี้

```
public interface Runnable{  
    public abstract void run();  
}
```

ชนิดของอินเทอร์เฟส

* หน้า 15

- **Interface with Default Methods** คือ อินเทอร์เฟสที่มี default methods ซึ่งเป็นเมธอดที่มีการกำหนดการทำงานไว้แล้ว คลาสที่ implements อินเทอร์เฟสนี้สามารถเลือกที่จะโอเวอร์ไรด์หรือใช้การทำงานเริ่มต้นที่กำหนดไว้ในอินเทอร์เฟสก็ได้

```
public interface Vehicle {
    public abstract void startEngine();
    public default void stopEngine() { // Default method
        System.out.println("Engine is stopped.");
    }
}

public class Car implements Vehicle {
    @Override
    public void startEngine() {
        System.out.println("Car engine started.");
    }
    // เมธอด stopEngine ไม่จำเป็นต้องโอเวอร์ไรด์ เนื่องจากมีการกำหนดในอินเทอร์เฟสแล้ว
}

public class Motorcycle implements Vehicle {
    @Override
    public void startEngine() {
        System.out.println("Motorcycle engine started.");
    }
    @Override // โอเวอร์ไรด์เมธอด stopEngine ด้วยการทำงานที่แตกต่างไป
    public void stopEngine() {
        System.out.println("Motorcycle engine is now off.");
    }
}
```


ชนิดของอินเทอร์เฟส

* ห้ามใช้

- **Interface with Static Methods** คือ อินเทอร์เฟสที่มี เมธอดของอินเทอร์เฟส (static methods) ที่มีการกำหนดการทำงานไว้แล้ว ซึ่งไม่จำเป็นต้องมีการ implements ในคลาสที่ใช้งานอินเทอร์เฟส

```
public interface Calculator {  
    public abstract double add(double a, double b);  
    public abstract double subtract(double a, double b);  
    // Static method in interface  
    public abstract static double multiply(double a, double b) {  
        return a * b;  
    }  
}  
  
public class SimpleCalculator implements Calculator {  
    @Override  
    public double add(double a, double b) {  
        return a + b;  
    }  
  
    @Override  
    public double subtract(double a, double b) {  
        return a - b;  
    }  
}
```

ชนิดของอินเทอร์เฟส

- **Interface with Constants** แม้จะไม่ใช่ที่นิยมในการออกแบบโปรแกรม แต่อินเทอร์เฟสสามารถถูกใช้เพื่อกำหนดค่าคงที่ ซึ่งค่าเหล่านี้จะกำหนดให้เป็น public static final โดยค่าเริ่มต้น

```
public interface Config {  
    // Constants in interface  
    public static final int MAX_WIDTH = 1024;✓  
    public static final int MAX_HEIGHT = 768;✓  
}
```

การใช้งานค่าคงที่จากอินเทอร์เฟส

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Max Width: " + Config.MAX_WIDTH);  
        System.out.println("Max Height: " + Config.MAX_HEIGHT);  
    }  
}
```

ชนิดของอินเตอร์เฟส

- **Marker Interface** คือ อินเตอร์เฟสที่ไม่มีเมธอดและไม่มีแอททริบิวต์มีรูปแบบดังนี้

```
public interface Serializable {  
  
}  
  
public interface Cloneable {  
  
}
```

การใช้ Marker Interface เป็นวิธีการในการทำ meta-programming โดยใช้งานในลักษณะของ "การแท็ก" หรือ "การมาร์ก" คลาสเพื่อบอกใบ้หรือระบุบางสิ่งบางอย่าง เช่น

- **Serializable Interface:** ใน Java, java.io.Serializable เป็นตัวอย่างของ Marker Interface ที่ใช้เพื่อระบุว่าคลาสนั้นสามารถถูก serialize ได้ ซึ่งหมายความว่าอ็อบเจกต์ของคลาสนั้นสามารถถูกแปลงเป็นลำดับของไบนารีและสามารถถูกบันทึกหรือส่งผ่านเครือข่ายได้
- **Cloneable Interface:** java.lang.Cloneable อีกหนึ่งตัวอย่างของ Marker Interface ที่ใช้เพื่อระบุว่าคลาสนั้นสามารถถูกโคลน (clone) ได้ หมายความว่าอนุญาตให้สร้างสำเนาของอ็อบเจกต์จากคลาสนั้นๆ ผ่านเมธอด clone() ของคลาส Object

หัวข้อ

- เมธอด Constructor
- คีย์เวิร์ด this และ super และเมธอด this() และ super()
- คลาส (Class)
 - คลาสสมบูรณ์ (Class)
 - คลาสไม่สมบูรณ์ (Abstract class)
 - คลาสภายใน (Inner class หรือ Nested class)
- อินเตอร์เฟส (Interface)





THANKS !